INSTITUTO CERVANTES

Carrera Analista Programador · Cátedra Práctica Profesionalizante

DOCUMENTACIÓN TÉCNICA

Explicación del código

Qué hace cada carpeta, archivo y pieza de la plataforma ENTS, y cómo se conectan entre sí

PROYECTO ENTS — Peticiones vecinales	**DOCENTE** Ing. Loza	**DIRECTOR DEL PROYECTO** Andrada, Juan Pablo
REPOSITORIO github.com/andradaj09/ENTS	**TAMAÑO** ≈ 60 archivos de código · 14 tablas	**SITIO PUBLICADO** entscordoba.netlify.app

AUTORES

Andrada, Juan Pablo
Legajo 27318

Patrón, Máximo
Legajo 27311

Jiménez, Ignacio
Legajo 28177

Contenido

1 Panorama general
2 Archivos de la raíz
3 Carpeta public/
4 Arranque y estructura de src/
5 Páginas (src/pages)
6 Componentes (src/components)
7 Servicios (src/services)
8 Hooks y utilidades
9 Estilos (src/styles)
10 Base de datos (supabase/schema.sql)
11 Documentación (docs/)
12 Cómo se conecta todo: crear una petición
13 Cosas sobrantes detectadas

GUÍA DE LECTURA

Qué es este documento. Un recorrido por todo el código de ENTS: cada carpeta, cada archivo y qué responsabilidad cumple dentro del sistema.

Para qué sirve. Para que cualquier integrante o docente entienda dónde está cada cosa y cómo fluye la información, sin tener que leer el código línea por línea.

Cómo leerlo. Va de lo general a lo particular: primero la arquitectura, después cada carpeta y al final un ejemplo completo de cómo trabajan juntas todas las piezas.

1 Panorama general

ENTS es una aplicación web organizada en **tres capas**:

VISTA

Lo que se ve

- `src/pages`
- `src/components`
- `src/styles`

Pantallas en React que el navegador descarga desde Netlify.

navegador → Netlify (HTTPS)

CONTROLADOR

La lógica que conecta

- `src/services`
- `src/hooks`
- `src/utils`

Piden y mandan datos a la base; cálculos que corren en el navegador.

servicios → Supabase (API + RPC)

MODELO

Datos y reglas

- `supabase/schema.sql`
- PostgreSQL en Supabase

Tablas, permisos, cálculos automáticos y avisos.

disparadores → notificaciones

Decisión central del proyecto: las reglas importantes viven en la base de datos, no en la pantalla. Quién puede ver o borrar algo, el score de urgencia, los avisos y la elevación a las 50 firmas los decide PostgreSQL, así nadie puede saltarlas manipulando el navegador. La pantalla solo esconde lo que la base igual rechazaría.

2 Archivos de la raíz (configuración)

Archivo	Qué hace
`package.json`	Lista de dependencias y comandos: `npm run dev` (desarrollo), `build` (versión de producción), `test` (pruebas) y `lint` (revisión del código).
`package-lock.json`	Versiones exactas instaladas de cada dependencia (lo genera npm automáticamente).
`vite.config.js`	Configura Vite, la herramienta que compila la aplicación, con el plugin de React.
`index.html`	La única página HTML. Carga la fuente Plus Jakarta Sans y el manifiesto de la app, y trae un pequeño script que aplica el modo oscuro *antes* de dibujar (así no parpadea). En `<div id="root">` se monta React.
`netlify.toml`	Le indica a Netlify cómo compilar (`npm run build` , Node 22); hace que cualquier ruta (`/mapa` , `/peticiones/12`) sirva `index.html` , porque las rutas las maneja React; y evita guardar en caché `sw.js` .
`.oxlintrc.json`	Reglas del revisor automático de código (oxlint), por ejemplo las reglas de los hooks de React.
`.env.example`	Plantilla de variables de entorno: URL de Supabase y su clave pública. El `.env` real nunca se sube al repositorio.
`.gitignore`	Lo que Git ignora: `node_modules` , `dist` , `.env` , etc.
`README.md`	Explicación del proyecto y pasos para instalarlo y ejecutarlo.

3 Carpeta public/ (archivos que se sirven tal cual)

Archivo	Qué hace
`manifest.webmanifest`	Convierte el sitio en una app instalable: nombre "ENTS", colores, íconos y accesos directos a "Nueva petición" y "Mapa vecinal".
`sw.js`	El *service worker*, un programa en segundo plano. Guarda en caché los archivos de la app para que abra aunque falle la red: las páginas se piden primero a la red y los archivos con hash (`/assets`) se toman de la caché. Los datos de Supabase nunca se guardan en caché.
`logo.svg` / `favicon.svg`	El logo de anillos concéntricos.
`icons/`	Íconos PNG para la app instalada en el celular.

4 Arranque y estructura de src/

Archivo	Qué hace
`main.jsx`	Punto de entrada. Carga Bootstrap y los estilos globales, registra el service worker, empieza a escuchar el aviso de "app instalable" y monta `<App>` dentro de `<AuthProvider>`.
`supabase/client.js`	Crea la única conexión con Supabase usando las variables del `.env`. Todos los servicios la importan.
`contexts/AuthContext.jsx`	Guarda la sesión y la comparte con toda la app: `user` (la cuenta), `profile` (nombre, rol, área…) y `loading`. Escucha cuando alguien entra o sale y, si llega un enlace de recuperación de contraseña, lleva a `/restablecer`. Se lee con `useAuth()`.
`App.jsx`	El mapa de rutas (React Router). Rutas públicas: `/login`, `/registro`, `/recuperar`, `/restablecer` y `/privacidad`. El resto exige sesión. Casi todas las páginas se cargan "perezosas" (`lazy`): se descargan recién al visitarlas.
`routes/ProtectedRoute.jsx`	Si no hay sesión, manda al login y recuerda a qué página se quería ir.
`routes/PanelRoute.jsx`	Si el rol no tiene permiso para ver ese panel (según la tabla de permisos), manda al inicio.
`layouts/MainLayout.jsx`	El marco de la app: barra superior (logo, campana, avatar), menú lateral fijo armado según los permisos de cada rol, fondo animado y el espacio donde aparece cada página. En el celular, el menú se abre como un cajón. En el primer ingreso muestra la ventana "¿De qué barrio sos?".

5 Páginas (src/pages)

Cada archivo es una pantalla completa de la aplicación.

Archivo	Ruta	Qué hace	Quién lo ve
`AuthPage.jsx`	/login, /registro, /recuperar	Una pantalla con tres modos: ingresar (correo o Google), registrarse (vecino o funcionario) y pedir el correo de recuperación. Traduce los errores al español.	Público
`RestablecerPassword.jsx`	/restablecer	Elegir una contraseña nueva desde el enlace del correo.	Desde el correo
`Privacidad.jsx`	/privacidad	Política de privacidad (Google la exige para el ingreso con Google).	Público

Archivo	Ruta	Qué hace	Quién lo ve
`Home.jsx`	/	Inicio: saludo, accesos rápidos, tarjeta "Tu barrio", novedades y próximos eventos, y las peticiones ordenadas por prioridad, agrupadas en mi barrio, cerca y resto de la ciudad, con filtro por categoría.	Todos
`Dashboard.jsx`	/dashboard	"Mis solicitudes": las peticiones propias en cualquier estado.	Según permisos
`CreatePetition.jsx`	/peticiones/nueva	Formulario de petición: sugiere categoría y barrio, muestra peticiones parecidas, avisa datos personales o insultos antes de enviar, sube fotos comprimidas y lleva al detalle.	Con permiso de crear
`PetitionDetail.jsx`	/peticiones/:id	Detalle: fotos, firma, prioridad y meta de 50, línea de seguimiento, comentarios y score de urgencia. Autor y firmantes solo para quien puede verlos; barra de editar o borrar para moderadores.	Todos
`Mapa.jsx`	/mapa	Mapa con dos vistas: marcadores por petición, o barrios coloreados por cantidad de peticiones, firmas o urgencia. Filtro por barrio y lista lateral. Abre centrado en el barrio del vecino, con su contorno destacado.	Según permisos
`Comunidad.jsx`	/comunidad	Muro vecinal anónimo (publicaciones positivas o negativas por barrio), búsqueda y peticiones del barrio. Arranca filtrada por el barrio del vecino.	Según permisos
`Eventos.jsx`	/eventos	Agenda de próximos eventos (primero los del barrio del vecino, con filtro "Mi barrio"); formulario para crear y borrar si se tiene permiso.	Según permisos
`Estadisticas.jsx`	/estadisticas	Números públicos y anónimos: peticiones por categoría, barrio, semana y urgencia, y ranking de barrios más activos y que más resolvieron, con gráficos y tablas.	Según permisos
`Perfil.jsx`	/perfil	Nombre, foto o avatar predefinido, "Mi barrio", barrios seguidos, insignias y "lo que podés hacer" según el rol.	Según permisos
`Bandeja.jsx`	/bandeja	Peticiones del área del funcionario por prioridad, filtros por etapa y resumen del área (sin resolver, urgentes, elevadas, días promedio).	Funcionario y admin

Archivo	Ruta	Qué hace	Quién lo ve
`AdminPetitions.jsx`	/admin/peticiones	Moderación: resumen de lo pendiente, peticiones por aprobar en filas compactas con miniaturas (primero las marcadas), solicitudes de funcionario y comentarios denunciados u ocultos.	Moderador y admin
`Usuarios.jsx`	/admin/usuarios	Directorio con datos completos, búsqueda, filtros por rol, asignación de área y cambio de rol (este último, solo el dueño).	Admin
`EstadisticasAdmin.jsx`	/admin/estadisticas	Panel analítico: cuentas, registros por semana, moderación, firmas por día y usuarios más activos; exporta a Excel y PDF.	Admin
`Documentos.jsx`	/documentos	Documentos privados del trabajo práctico (PDF): abrir, subir y borrar.	Dueño (o a quien habilite)
`Permisos.jsx`	/permisos	Matriz de rangos por paneles con interruptores de Ver y Modificar.	Solo el dueño

6 Componentes (src/components)

Piezas reutilizables que forman las páginas.

Archivo	Qué es
`Avatar.jsx`	Foto, avatar predefinido o iniciales, dentro del anillo del color del rol; etiqueta "CEO" para el dueño.
`avatares.js`	Avatares geométricos predefinidos (se guardan como `preset:N`).
`MenuUsuario.jsx`	Menú del avatar: datos de la cuenta, perfil, mis solicitudes, instalar app, modo oscuro y cerrar sesión.
`NotificacionesMenu.jsx`	Campana con contador de no leídas y panel de avisos (firmas, hitos, respuestas, comentarios, barrios seguidos); las novedades del barrio propio van primero.
`PetitionCard.jsx`	Tarjeta de una petición: categoría, barrio, firmas, prioridad y urgencia.
`Seguimiento.jsx`	Línea de tiempo de 5 etapas con respuestas oficiales; formulario para quien puede responder.
`Comentarios.jsx`	Comentarios anónimos: escribir, denunciar y borrar; los moderadores ven autor, alertas y denuncias. Si el autor lo permite, muestra su barrio.
`EditarPeticion.jsx`	Ventana para que un moderador corrija una petición.
`PeticionesSimilares.jsx`	Mientras se escribe, busca peticiones parecidas para firmarlas en vez de repetirlas.
`SelectorFotos.jsx`	Elegir o arrastrar hasta 3 fotos y comprimirlas en el navegador.
`SelectorUbicacion.jsx`	Mapa chico para marcar dónde ocurre el problema.
`GaleriaFotos.jsx`	Fotos de la petición con visor a pantalla completa; en la moderación se muestran como miniaturas.
`Graficos.jsx`	Gráficos hechos a mano (barras, columnas, apiladas y tarjetas de números), cada uno con su versión en tabla para accesibilidad.
`BienvenidaBarrio.jsx`	Ventana del primer ingreso: "¿De qué barrio sos?", con la opción "Elegir después".
`ElegirBarrio.jsx`	Selector del barrio propio, a mano o con "Usar mi ubicación"; lo usan la bienvenida, el inicio y el perfil.
`TuBarrio.jsx`	Tarjeta del inicio con el resumen del barrio del vecino comparado con la ciudad.
`MiBarrioPerfil.jsx`	Sección "Mi barrio" del perfil: cambio una vez cada 30 días y opción de mostrar "Vecino/a de [barrio]" en los comentarios.
`BarriosSeguidos.jsx`	Elegir los barrios de los que se quieren recibir avisos.
`Insignias.jsx`	Logros por participar (primera petición, firmas, actividad en el barrio propio, etc.).
`OpcionesBarrios.jsx`	Opciones del selector de barrios agrupadas por localidad.
`Icon.jsx`	Todos los íconos, dibujados con líneas rectas.

Archivo	Qué es
`Logo.jsx`	El logo de anillos.
`FondoAnimado.jsx`	Figuras geométricas que flotan de fondo (quietas en celular, por rendimiento).
`HeroArt.jsx`	Ilustración del inicio.
`PalabraRotativa.jsx`	"Tu barrio, más cerca / más seguro / más limpio / más unido".

7 Servicios (src/services)

Ningún componente habla directo con Supabase: toda la comunicación pasa por estos tres archivos.

Archivo	Qué hace
`authService.js`	Cuentas: registro, ingreso con correo o Google, recuperación y cambio de contraseña, cerrar sesión, editar el perfil y subir el avatar. Traduce los errores de Supabase al español.
`petitionService.js`	Lo principal: peticiones (listar, ver, crear con fotos, editar, borrar, moderar), firmas, categorías y barrios (guardados en caché por sesión), notificaciones, eventos, muro, directorio de usuarios, solicitudes de funcionario, estadísticas y reportes. Pide solo las columnas visibles (no `*`) porque la columna del autor está oculta para preservar el anonimato.
`participacionService.js`	Seguimiento, comentarios, búsqueda de similares, barrios seguidos, logros, áreas, roles, documentos privados, permisos por panel y barrio del vecino (elegirlo, resumen y ranking). Casi todo llamando a funciones de la base (`rpc`).

8 Hooks y utilidades

8.1 Hooks (src/hooks)

Archivo	Qué hace
`useEsDueno.js`	Pregunta una vez a la base si la cuenta es la del dueño y lo recuerda.
`usePermisos.js`	Carga una vez qué paneles puede ver y modificar el rol, ofrece `puede('bandeja', 'modificar')` y se recarga cuando el dueño cambia un permiso.

8.2 Utilidades (src/utils): lógica pura, sin pantallas

Archivo	Qué hace
`clasificador.js`	Sugiere la categoría contando raíces de palabras clave; el título vale doble.
`barrios.js`	Detecta el barrio más cercano a la ubicación (distancia sobre la Tierra, hasta 3 km).

Archivo	Qué hace
`barrioVecino.js`	Recuerda si la persona eligió "Elegir después" y calcula los barrios cercanos (centro a menos de 3 km) para el inicio.
`zonasMapa.js`	Divide la ciudad en áreas por barrio (diagrama de Voronoi con `d3-delaunay`) y define la escala de azules del mapa.
`prioridad.js`	Los 5 niveles de prioridad por firmas, la meta de 50, el orden del inicio y las etapas de seguimiento.
`moderacion.js`	Detecta correos, teléfonos, DNI e insultos antes de enviar (la base repite el control).
`roles.js`	Nombres de los roles, quién ve identidades y el texto de "lo que podés hacer".
`paneles.js`	Lista de los 11 paneles con nombre, ruta, ícono y qué habilita "modificar".
`imagenes.js`	Comprime fotos a un máximo de 1600 px en JPEG (≈ 5 MB → ≈ 300 KB).
`ubicacion.js`	Centro de Córdoba, mapa base de OpenStreetMap y ubicación aproximada si la petición no trae coordenadas.
`format.js`	"Hace 2 h", números con punto de miles y umbrales de urgencia.
`exportar.js`	Genera los CSV que Excel abre bien (separador `;` y acentos).
`tema.js`	Modo claro u oscuro, guardado en el navegador.
`instalacion.js`	Botón "Instalar app" (Chrome) y ayuda para iPhone.

8.3 Pruebas automáticas (src/utils/__tests__)

49 pruebas con Vitest que verifican prioridad, clasificación, barrios, moderación de textos, áreas del mapa, exportación y roles. Se ejecutan con `npm test` antes de cada publicación.

9 Estilos (src/styles)

Archivo	Qué cubre
`theme.css`	La base de todo el diseño: colores, tipografía, barra superior, menú, tarjetas, botones, anillos de rol y ajustes de rendimiento en celular.
`oscuro.css`	El modo oscuro; se activa con `data-tema="oscuro"` en la página.
`auth.css`	Pantallas de acceso: login, registro, recuperación y privacidad.
`comunidad.css`	El muro vecinal.
`estadisticas.css`	Gráficos y paneles de estadísticas.
`mapa.css`	El mapa y sus marcadores.

10 Base de datos (supabase/schema.sql)

Esquema completo y reproducible de PostgreSQL, organizado en 20 secciones que se agregaron en orden a medida que creció el proyecto.

10.1 Tablas

Tabla	Qué guarda
`profiles`	Datos de cada usuario: nombre, rol, área, avatar y barrio donde vive (con la fecha del último cambio y si lo muestra en sus comentarios). Se crea sola al registrarse (disparador sobre la tabla de cuentas de Supabase).
`peticiones`	Pedidos: texto, fotos, ubicación, estado, etapa de seguimiento, score, cantidad de firmas, alertas y elevación.
`firmas`	Una firma por persona y petición.
`peticion_actualizaciones`	Respuestas oficiales del seguimiento.
`categorias` / `areas`	16 categorías agrupadas en las áreas del municipio.
`zonas`	Los 111 barrios de Córdoba y Villa Allende con su ubicación.
`comentarios` / `denuncias`	Comentarios anónimos y sus denuncias.
`notificaciones`	Los avisos de la campana.
`barrios_seguidos`	Barrios que sigue cada persona.
`publicaciones` / `eventos`	Muro vecinal y agenda.
`permisos_panel`	Qué ve y modifica cada rol en cada panel.

10.2 Disparadores (se ejecutan solos)

Disparador	Cuándo y qué hace
`preparar_peticion`	Al crear o editar una petición: fija la etapa inicial, detecta alertas, asigna el barrio si falta y calcula el score de urgencia.
`contar_firmas`	Al firmar o retirar la firma: mantiene el contador al día y recalcula el score.
`aviso_firma` / `avisos_peticion`	Avisan hitos de firmas, la elevación a las 50, los cambios de estado y las novedades en barrios seguidos.
`revisar_comentario`	Bloquea datos personales y oculta insultos hasta que un moderador los revise.
`aviso_comentario`	Avisa al autor de la petición cuando la comentan.
`contar_denuncias`	Con 3 denuncias, el comentario se oculta automáticamente.

10.3 Funciones

- **Esquema `privado`** (la app no puede llamarlas directamente): auxiliares como `calcular_urgencia`, `tiene_permiso`, `es_dueno`, `barrio_mas_cercano` y `revisar_texto`.
- **Esquema `public`** (las llama la app): se ejecutan con permisos elevados pero verifican adentro quién llama. Por ejemplo `mis_peticiones`, `identidades_peticion`, `actualizar_seguimiento`, `peticiones_similares` (compara textos por trigramas), `estadisticas_publicas`, `estadisticas_admin`, `cambiar_rol`, `guardar_permiso`, `mis_permisos`, `borrar_comentario`, `elegir_mi_barrio` (con el límite de un cambio cada 30 días), `resumen_mi_barrio` y `ranking_barrios`.

10.4 Seguridad

- **RLS (seguridad por fila)** en todas las tablas: cada una define quién puede leer, crear, editar o borrar cada fila.
- **Anonimato por columna:** la columna del autor de una petición no es legible para los vecinos; las identidades solo se obtienen mediante funciones que verifican el rol.
- **Archivos:** avatares, fotos de peticiones y documentos privados tienen sus propias reglas por carpeta.

11 Documentación (docs/)

Archivo	Qué hace
`entrega/entregables/datos.cjs`	Fuente única de datos: equipo, 20 actividades con ruta crítica, requisitos (22 RF y 8 RNF), 14 historias de usuario y glosario.
`entrega/entregables/generar.cjs`	Arma los 7 entregables y el documento combinado, y actualiza la documentación general con el mismo diseño, historial, requisitos, historias de usuario y glosario.
`entrega/entregables/estilos.css`	El diseño de los PDF (portada, índice, tablas, encabezados).
`entrega/documentacion.html`, `guion_defensa.html`	Documentación general del trabajo práctico y guion de la defensa, con sus PDF.
`Propuesta_mejoras_ENTS.pdf`	Documento con propuestas de mejora.

11.1 Datos de demostración (supabase/demo)

Archivo	Qué hace
`demo_cargar.sql`	Carga 300 cuentas ficticias (`@demo.ents.invalid`, sin contraseña), 200 peticiones con firmas, comentarios, publicaciones y eventos, siempre con los mismos valores para que el resultado sea reproducible.
`demo_borrar.sql`	Borra todos los datos de demostración sin tocar los reales.
`ilustraciones.cjs`	Genera las 32 ilustraciones que se usan como fotos de las peticiones de demostración.
`LEEME.md`	Cómo cargar y borrar los datos de demostración.

12 Cómo se conecta todo: crear una petición

Este recorrido muestra cómo trabajan juntas las piezas descritas, desde que un vecino escribe hasta que el municipio responde.

1. **Escribir.** En `CreatePetition.jsx`, mientras el vecino escribe, `clasificador.js` sugiere la categoría y `barrios.js` el barrio; `PeticionesSimilares` consulta `peticiones_similares` en la base y `moderacion.js` avisa si escribió un teléfono o un insulto.
2. **Fotos.** `SelectorFotos` y `imagenes.js` comprimen las fotos en el navegador.
3. **Enviar.** `petitionService.createPeticion` sube las fotos al almacenamiento de Supabase e inserta la petición.
4. **La base controla.** RLS verifica que sea propia, que quede "pendiente" y que el rol tenga permiso de crear. El disparador `preparar_peticion` calcula el score, las alertas y el barrio, y `avisos_peticion` avisa a los moderadores.
5. **Moderar.** Un moderador la aprueba en `AdminPetitions`: pasa a "publicada" y se avisa a quienes siguen ese barrio.
6. **Firmar.** Cada firma dispara `contar_firmas`, que recalcula el score y la prioridad; al llegar a 50, el área recibe el aviso de elevación.
7. **Responder.** El funcionario responde desde la `Bandeja` mediante `actualizar_seguimiento`, y el autor y los firmantes reciben el aviso en la campana.

13 Cosas sobrantes detectadas

Ya se quitaron la dependencia `react-bootstrap` (Bootstrap se usa solo con sus clases CSS), las imágenes sobrantes de la plantilla de Vite, estilos de una vista de barrios que ya no existe y funciones que nadie llamaba. Queda una sola cosa, que no afecta el funcionamiento:

Elemento	Situación
`privado.puede_ver_identidades()`	Función de la versión de 4 roles que dos funciones antiguas todavía usan. Funciona, pero convive con su reemplazo más nuevo por área (`puede_ver_identidades_de`).